claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_4b294c39-6a7\agent-ac99675a67165223d.jsonl`
- SHA-256: `1bdf0c3a564429c4bf97b566881b7acf146db4279dca997b26ba1570aeb26588`
- Source modified: `2026-06-22T18:54:27+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_4b294c39-6a7`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`

Transcript

1. user (2026-06-22T18:53:50.472Z)

You are an accuracy + completeness critic for a user-facing guide. Below are (A) the VERIFIED FACTS and (B) a DRAFT guide written from them. Find every place the draft: (1) states a command/endpoint/flag NOT supported by the facts (a fabrication ? the worst kind), (2) is missing a load-bearing step a user needs, (3) is confusingly worded for a non-technical user, or (4) contradicts the shipped async behavior (process + compile are BOTH async 202+poll now). Return a concise, prioritized fix list (each: what's wrong + the exact correction). If the draft is accurate + complete, say so explicitly.

A) VERIFIED FACTS:

packaging

All facts verified. Here is the report.

Engine downloadable-app: VERIFIED install + run surface

Repo: `C:/Users/avidu/Projects/bhi-computeval`. **Caveat on freshness:** the local checkout is on branch `feat/compute-provenance` (HEAD `ee6fba8`) and `git pull --ff-only` FAILED (the configured upstream ref `refs/heads/feat/compute-provenance` was not fetched). I verified against this local checkout read-only; it may lag the remote. All citations below are from this tree.

Exact install commands (VERIFIED)

Two paths exist. Note the package is **not yet on PyPI** ? README says "(once published)" and `pyproject.toml:18` pins `version = "0.1.0"`, so today install is from a local checkout (`.`).

1. **uv (preferred)** ? `uv tool install badminton-highlight-indexer` (once published) or `uv tool install .` (README.md:42-43). uv drops `indexer` into an isolated tool-bin that is NOT auto-added to PATH; run `uv tool update-shell` once (README.md:44; `scripts/install.py:181-185`).
2. **Installer script** ? `python scripts/install.py` (README.md:48). Flags: `--method {auto,uv,pip}` (default auto = uv-if-present else pip), `--target` (default `.`, or a PyPI spec), `--dry-run` (`scripts/install.py:141-146`). It (a) runs `uv tool install <target>` or `pip install <target>` (`install.py:125-129`), (b) writes a per-OS launcher shortcut into the OS app-data dir that pins `RALLY_APP_HOME` then runs `indexer --app` (`install.py:62-93,189-194`), (c) copies `uninstall.py` next to the data and writes `uninstall_manifest.json` (`install.py:196-211`).
3. **Dev/source path** ? `pip install -e .` (runtime) or `pip install -e .[dev]` (README.md:83-85); registers the `indexer` console script via `[project.scripts] indexer = "backend.main:main"` (`pyproject.toml:84-86`).

The console entry point `indexer` maps to `backend/main.py::main` (`pyproject.toml:86`).

Exact run command (VERIFIED)

- `indexer --app` ? boots the local FastAPI server **localhost-only (127.0.0.1)**, auto-picks a free port if `--port` (default 8000) is busy, and opens the bundled SPA in the default browser once `/api/health` is live (`main.py:1138-1142,1216-1221,1349-1364`; port fallback `_resolve_app_port` at `main.py:155-157`; browser-open at `main.py:160-182,1363`). Set `RALLY_NO_BROWSER=1` to suppress the browser open (`main.py:164-165`).
- `indexer --server` ? same server but bind address is config/profile-driven (default 127.0.0.1; 0.0.0.0 only when `security.edge_auth_enabled`) and does NOT seed config or auto-open a browser (`main.py:1365-1373`; `_resolve_bind_host` at `main.py:119-132`).
- Other entry modes: `indexer --setup` (runs `scripts/doctor.py`, `main.py:1156-1168`), `indexer --video-path <f>` (CLI single-shot, `main.py:1268`), `--debug-clip`, `--trim-start/--trim-end`.

What `--app` does on first run (VERIFIED)

1. **Seeds a default config** if none exists: `write_light_default_config()` writes `LIGHT_DEFAULT_CONFIG` to `default_config_path()` = `<app-home>/config.json`, **never overwriting** an existing file (`main.py:1216-1221`; `config/loader.py:231-246`). That config is `{"sport":"badminton", "deployment":{"profile":"truly-local"}, "indexing":{"default_segmenter":"motion","use_gpu":false}}` (`config/loader.py:224-228`) ? i.e. detector `motion`, offline, no API key/GPU/cloud. This seeding is scoped to `--app` ONLY; `--server` and CLI are byte-identical-no-seed (`main.py:1214-1216` comment).
2. **Serves the bundled SPA single-origin.** `backend/ui/` is the committed snapshot; `/` serves `backend/ui/index.html` and `/assets` is mounted from `backend/ui/assets/` (`main.py:198-216`). Verified present: `backend/ui/{index.html, assets/, ui_version.json}` (spa_commit `ed33a51`, built 2026-06-22, `engine_api_version: 1`). Falls back to the legacy in-engine dashboard only if no bundled UI is found (`main.py:204`).
3. Acquires a single-instance lock keyed to the DB path (refuses a 2nd instance, `main.py:1242-1257`) and sweeps stale jobs from a prior crash (`main.py:1263-1265`).

First-run wizard (VERIFIED ? but lives in the SPA, not the engine)

The 3-step setup wizard is **client-side React in the bundled SPA**, not Python. The engine only exposes the data it renders from: `GET /api/capabilities` (`main.py:347-381`) reports segmenters, default segmenter, ffmpeg/ffprobe presence+paths, GPU heuristic (`nvidia-smi` on PATH), `gemini_key_present` (boolean only, never the value), WASB-weights presence, and the deployment/cloud-availability. The wizard JS is bundled in `backend/ui/assets/index-ukuS9wxa.js` (verified it references the wizard + consumes `capabilities`). The "show once per browser" gating is client-side (browser storage), not engine-enforced. So: the wizard ships and is served by `indexer --app`, but it is the khelsutra SPA snapshot ? there is no Python/CLI wizard.

Prerequisites (VERIFIED)

- **Python ? 3.10** (`pyproject.toml:21`).
- **FFmpeg + FFprobe required**, and **intentionally NOT redistributed** (GPL `libx264`) ? the user must install it (README.md:61,67-74). The engine resolves binaries via `RALLY_FFMPEG` / `RALLY_FFPROBE` env vars, else bare `ffmpeg`/`ffprobe` on PATH (`backend/utils/ffmpeg.py:25-32`). `require_ffmpeg()` raises with an OS-specific hint (`ffmpeg.py:43-62`): Windows `winget install Gyan.FFmpeg`, macOS `brew install ffmpeg`, Linux `sudo apt-get install -y ffmpeg`. `indexer --setup` ? `scripts/doctor.py` checks ffmpeg/ffprobe and prints the same install hints (`doctor.py:89-113`).
- **LIGHT tier is torch-free.** `torch`/`torchvision` are deliberately excluded from `pyproject.toml` (need `--index-url`); the `[gpu]` extra is intentionally empty/documentation-only (`pyproject.toml` `gpu = []` block). Native-WASB GPU path is separate opt-in provisioning, not part of the default download (README.md:65).

Where data/config/env live (VERIFIED)

Resolved by `backend/utils/app_home.py`:
- `RALLY_APP_HOME` set ? that dir is the app home; **unset ? `Path(".")` (CWD)**, byte-identical to pre-packaging behavior (`app_home.py:36-45`). The packaged launcher shortcut pins

`RALLY_APP_HOME` to the OS app-data dir (`install.py:70-93`).
- OS defaults (`default_os_app_home`, `app_home.py:101-116`): Windows `%APPDATA%\Khelsutra`, macOS `~/Library/Application Support/Khelsutra`, Linux `$XDG_DATA_HOME/Khelsutra` (else `~/.local/share/Khelsutra`).
- `config.json` ? `<app-home>/config.json` (`app_home.py:53-58`); `.env` ? `<app-home>/.env`, loaded at import only when `RALLY_APP_HOME` is set AND the file exists (`app_home.py:61-78`; `main.py:25`); output dir (DB + reels) roots under app-home when the env is set (`resolve_output_dir`, `app_home.py:81-98`; applied at `main.py:1233-1236`).

Uninstall (VERIFIED)

`python uninstall.py` (the copy dropped in the app-home) reverses the install from the manifest: uninstalls the package (`uv tool uninstall` / `pip uninstall -y`, matching install method), removes installer-created files, and **keeps user data by default**; `--purge` also deletes `config.json`/`.env`/`output/` (`scripts/uninstall.py:1-16,96-172`). A containment guard (`_within`, `uninstall.py:68-79`) refuses to delete anything outside the app-home.

Gotchas (VERIFIED)

1. **Not on PyPI yet** ? `uv tool install badminton-highlight-indexer` won't resolve today; use `uv tool install .` / `python scripts/install.py` from a checkout (README.md:43 "once published").
2. **`indexer` may not be on PATH after `uv tool install`** ? needs `uv tool update-shell` + a new shell; the installer bakes the resolved absolute `indexer` path into the launcher to work around this, and warns loudly if it can't resolve it (`install.py:178-219`).
3. **FFmpeg is a manual prerequisite** ? the README itself flags ffmpeg-acquisition UX as still-manual and the packaging plan lists it as remaining OWNER work (PACKAGING_PLAN.md:next-session §, item 4). There is no auto-download of ffmpeg.
4. **`RALLY_APP_HOME` must be set BEFORE the process starts** to root state in the app-data dir ? the launcher shortcut does this; running bare `indexer --app` from a shell without it uses the CWD as app-home (`install.py:8-12` comment; `app_home.py:39-45`).
5. **`--app` is always localhost-only**; network exposure requires `--server` behind an edge-auth gate (`main.py:1351-1353,1369-1373`).
6. **Single-instance lock per DB** ? a 2nd `--app`/`--server` against the same output dir refuses to start (`main.py:1253-1257`).

NOT implemented / not shipped (per this checkout)

- No PyPI distribution yet (version 0.1.0, "once published").
- No bundled/auto-acquired ffmpeg ? manual install only.
- No native-WASB or torch in the default download; GPU and cloud tiers are separate opt-in provisioning (README.md:65; PACKAGING_PLAN.md §0/L10 north-star, still future + counsel/owner-GPU-gated).
- The "first-run wizard" is the SPA's client-side `SetupWizard`, not a Python/CLI wizard; the engine only serves it + the `/api/capabilities` data.

Key files: `C:/Users/avidu/Projects/bhi-computeval/backend/main.py`, `.../scripts/install.py`, `.../scripts/uninstall.py`, `.../scripts/doctor.py`, `.../backend/utils/app_home.py`, `.../backend/utils/ffmpeg.py`, `.../backend/config/loader.py`, `.../pyproject.toml`, `.../backend/ui/` (bundled SPA), `.../README.md`, `.../docs/PACKAGING_PLAN.md`.

---

api
I have all the verified facts I need. Here is the report.

VERIFIED reel-creation API flow ? `bhi-computeval/backend/main.py`

Provenance of this report: local checkout on branch `feat/compute-provenance` (HEAD `ee6fba8`). Note: `/api/compile` async (#329, `8bcf6ce`) and the compute-provenance block are

ALREADY on `origin/master` (`d92cc10` #341). The branch this was read on is one commit ahead with `skip_ai_handoff` forwarding. All citations are `backend/main.py` unless noted.

The chain (upload/gs:// ? process ? query ? compile ? download)

A client either (a) uploads a local file via the chunked-upload endpoints to get a server-local path, OR (b) passes a `gs://` / Drive URI directly. That path/URI feeds `/api/process` (async) ? poll `/api/jobs/{id}` until `result.play_segments` exist ? `/api/query` to filter segments ? `/api/compile` (async) ? poll the job for `result.download_url` ? GET that URL to download the reel.

---

1. `GET /api/capabilities` ? `main.py:347-381`

First-run wizard probe. No request body. Response (verified fields):
- `api_version` (int, `API_VERSION = 1`, `main.py:330`), `engine_version` (str)
- `segmenters`: `sorted(segmenter_registry.list_available().keys())` ? see registry section
- `default_segmenter`: from `indexing.default_segmenter`
- `ffmpeg`: `{ffmpeg: bool, ffprobe: bool, ffmpeg_path, ffprobe_path}`
- `gpu`: `{nvidia_smi_present: bool}` (heuristic ? `shutil.which("nvidia-smi")`, deliberately NOT `torch.cuda`)
- `gemini_key_present: bool`, `wasb_weights_present: bool`
- `deployment`: `{profile, compute_target, cloud_available}` ? `cloud_available` (`_cloud_available()`, `main.py:489-503`) gates whether the SPA offers the cloud compute toggle.

2. `POST /api/process` ? `main.py:772-821` ? **ASYNC, status_code=202** (VERIFIED)

Request body `ProcessRequest` (`main.py:262-273`):
- `video_path: str` (required ? local path OR `gs://`/Drive URI)
- `player_pool: Optional[List[str]]`, `max_frames: Optional[int]`, `segmenter: Optional[str]`, `locale: Optional[str]`, `compute: Optional[str]` (`"local"`/`"cloud"`/None)

Fast-fail at submit (immediate 4xx, before queuing): edge-auth 401 (`:786`), bad path/unknown URI scheme ? 400 (`:795`), missing LOCAL file ? 404 (`:803-804`), unknown `segmenter` ? 400 (`:805-810`). Then creates a `queued` job and submits the drain (`:812-817`).

Response (202): `{"job_id": <hex>, "status": "queued", "poll": "/api/jobs/{job_id}"}` (`:818-821`). The slow work (MD5 hash, validate, segment) runs on the worker via `_execute_processing` (`:622-769`).

3. `GET /api/jobs/{job_id}` ? `main.py:850-869` (VERIFIED)

404 on unknown id. Returns `{job_id, status, progress, video_id, error, result, reports, created_at, started_at, finished_at}`. `status` = execution state (`queued|running|done|failed`, `:853`); the pipeline verdict is **`result["status"]`** (`success|failed`). On a successful process job, `result` carries `play_segments` (`:736-742`). (Related: `GET /api/jobs/{job_id}/cost` `:824-836`, `GET /api/videos/{video_id}/cost` `:839-847`.)

4. `POST /api/query` ? `main.py:907-916` (VERIFIED ? still SYNC, returns 200)

Request body `QueryRequest` (`main.py:275-280`): `video_id: str` (required), optional `server`, `receiver`, `duration_min: float`, `event_type`. Response: `{"play_segments": [...]}` ? each segment has `id`, `start_time`, `end_time`, `metadata`. The client picks `id`s here to feed compile.

5. `POST /api/compile` ? `main.py:967-998` ? **ASYNC, status_code=202** (VERIFIED, #329)

Request body `CompileRequest` (`main.py:282-288`): `video_id: str`, `segment_ids: List[int]`, `output_filename: str`, optional `locale: str` (localizes the watermark text + font).

Fast-fail at submit via `_resolve_compile` (`:927-964`): edge-auth 401 (`:980`); video missing ? 404; unsafe filename ? 400; no matching segments ? 400; all segments below `stitching.min_shot_count` floor ? 400. Then creates a `kind='compile'` job (`:990-992`) and submits the drain.

Response (202): `{"job_id", "status":"queued", "poll":"/api/jobs/{job_id}"}` (`:995-998`). The

ffmpeg stitch runs on the worker in `_execute_compile` (`:1001-1042`); on `done`, the job `result` is `{"status":"success", "filepath", "download_url", "video_id"}` (`:1042`) where `download_url = /api/highlights/{video_id}/{out_name}` (`:1040`). The async move was specifically to kill the Cloudflare 524 from a >100s synchronous stitch (`:969-976`).

6. `GET /api/highlights/{video_id}/{filename}` ? `main.py:883-904` (VERIFIED)

Streams the compiled reel (FileResponse, `video/mp4` or `video/quicktime`). `filename` = the bare name passed to `/api/compile`. 400 on unsafe name, 404 if the video record or the compiled file is missing. (Sibling: `GET /api/reels` `:398-426` lists reels flat; `GET /api/reels/{filename}` `:429-446` downloads by bare name ? note `/api/reels` does NOT track a video_id?reel association, it's a global flat list, `:404-405`.)

Chunked upload (`backend/api/uploads.py`, router included at `main.py:876-878`)

Four endpoints (VERIFIED, `uploads.py`):
- `POST /api/upload/init` (`:64`) ? body `{filename, total_size_bytes?}`. 400 on bad name/disallowed ext (default `mp4,mov`, `:50`), 413 over `max_upload_mb` (default 4096). Returns `{upload_id, max_part_mb, next_part: "/api/upload/{id}/part/0"}`.
- `PUT /api/upload/{upload_id}/part/{index}` (`:89`) ? raw bytes body; parts STRICTLY sequential (409 on out-of-order, `:97-99`); 413 over per-part (`max_part_mb`, default 95) or total limit. Returns `{upload_id, received_parts, received_bytes}`.
- `POST /api/upload/{upload_id}/complete` (`:123`) ? body `{total_parts}`. 409 on part-count mismatch. Returns **`{path, video_id, size_bytes, next: "POST /api/process with this path"}`** ? `path` is the absolute server path you feed into `/api/process`.
- `DELETE /api/upload/{upload_id}` (`:147`) ? abort + delete partial.

Upload state is **in-process / in-memory** (`_uploads` dict, `uploads.py:28`) ? a server restart aborts partial uploads by design (single-box alpha, `uploads.py:6-8`).

Available segmenters (registry / default)

`segmenter_registry` (`backend/pipeline/segmenters/base.py:35-63`). Registration is triggered by importing the modules in `backend/pipeline/segmenters/__init__.py` (all six imported there). Registry keys = each class's `name` property (VERIFIED values):
- `motion` (`motion.py:14`, pure-OpenCV LIGHT-tier), `gemini` (`gemini.py:65`), `yolo_hybrid` (`yolo_hybrid.py:51`), `tracknet_hybrid` (`tracknet_hybrid.py:27`), `wasb_hybrid` (`wasb_hybrid.py:29`), `fusion_hybrid` (`fusion_hybrid.py:56`).

`/api/capabilities` returns `sorted(...keys())` of whatever registered at runtime ? modules whose heavy deps (torch/weights) fail to import won't appear, so the served list reflects actual box capability.

Default segmenter ? two distinct values (VERIFIED, important nuance):
- Pydantic model default: `default_segmenter = "yolo_hybrid"` (`backend/config/models.py:128`).
- BUT the LIGHT / batteries-included first-run seeded config writes `default_segmenter: "motion"` (`backend/config/loader.py:227`) ? the torch-free default an installed app actually gets.

The effective default in `_execute_processing` falls back to `"motion"` if `indexing` is unset (`main.py:652`). A per-request `segmenter` overrides it (`main.py:653`).

Job `result['compute']` provenance block (VERIFIED, the compute-provenance work)

Every process job result carries a `compute` block proving which backend ran the DETECT:
- **In-process default path**: stamped in the `finally` of `_execute_processing` (`main.py:744-753`): `{"path": "in_process", "requested": req.compute}`. `path` is `"not_run"` if nothing ran (validation/config fail).
- **Cloud path** (`_maybe_dispatch_remote`, `main.py:506-619`): on success returns `{"path":"cloud_run", "requested", "job", "region", "execution", "outputs_uri"}` (`:606-613`) ? `execution` is the real Cloud Run execution name (cross-checkable via `gcloud run jobs executions describe`). On failure: `{"path": "cloud_run" if ran else "not_run", "requested", "target":"cloud_run", "dispatched": ran}` (`:527-530`) ? `dispatched` honestly distinguishes a remote run that failed from a pre-dispatch config reject. A `[COMPUTE]` log line is also emitted (`:614`, `:755-757`).

The `compute` picker: per-request `compute="local"` forces in-process even on a cloud server (`:540-541`); `"cloud"` forces dispatch but `_failed`s with 400-ish job result if `_cloud_available()` is false (`:542-546`). Default-OFF: `get_compute_backend` returns `None` for any non-`cloud_run` target, so the in-process path is byte-identical (`:550-551`).

NOT implemented / honest caveats

- **`/api/compile` is NOT synchronous** anymore ? it returns 202 + job_id; a client still expecting a synchronous reel response is broken (#329).
- **`/api/process` does NOT return segments inline** ? only `{job_id}`; segments arrive via the polled job result.
- **Cloud path drops `player_pool` / `max_frames`** ? not threaded to the worker `infer` CLI yet (documented follow-up, `main.py:573`, `:701-702`).
- **`/api/reels` has no per-video association** ? reels are stored flat under the bare compile filename; there is no video_id?reel DB record (`main.py:404-405`).
- **Upload state is non-durable** (in-memory) ? partial uploads die on restart, and per-user partitioning of `upload_dir` is deferred to PR-3 (`uploads.py:6-8`).

---

cloud

All 23 tests pass. I have verified every component. Let me give the final report.

Compute Picker + Cloud Path ? VERIFIED state (engine repo `bhi-computeval`)

Branch caveat (read first): The cloud-picker provenance work is on branch `feat/compute-provenance`, **2 commits ahead of `origin/master`, NOT yet merged** (`4b6a560` compute-path provenance, `ee6fba8` forward `skip_ai_handoff`). The picker/dispatch core (`_maybe_dispatch_remote`, `_cloud_available`, `get_compute_backend`) and the `cloud-serving` profile predate this branch, but the `compute` field on `ProcessRequest`, the `cloud_available` capability, the `execution` provenance, and `scripts/verify_compute_path.py` live on this unmerged branch. The 23-test `tests/test_api_cloud_dispatch.py` suite passes locally. Tree also has an untracked `cloudbuild.cr.yaml`.

Reality check on "shipped": the dispatch shim + image + tests are landed and **mock-tested only**. The real Cloud Run L4 build/run (M7) is an **owner step that has NOT happened** ? `docs/COMPUTE_DECOUPLED_SERVING/runlogs/` contains only `RUNLOG_truly-local_TEMPLATE.md` and the README, no `DEPLOY_REPORT_cloud-serving_*`. The real GCP client (`GoogleCloudRunJobClient.run_job`) is `# pragma: no cover` ? never exercised in CI.

How a request picks cloud vs local

`ProcessRequest.compute: Optional[str]` (`backend/main.py:273`), values `"local" | "cloud" | None`. Resolved in `_maybe_dispatch_remote` (`backend/main.py:506`), called from `_execute_processing` at `backend/main.py:703`:
- `"local"` ? `return None` ? forces in-process even on a cloud server (`main.py:540-541`).
- `"cloud"` ? guarded by `_cloud_available()`; if false, returns a clear `_failed(...)` with `path="not_run"` (no silent local fallback) (`main.py:542-546`); if true, `get_compute_backend(global_config, target_override="cloud_run")` (`main.py:547`).
- `None` ? server default `get_compute_backend(global_config)` (`main.py:549`), default-OFF (None unless `compute_target=="cloud_run"`).
- unknown value (e.g. `"banana"`) ? warns and falls back to server default (`main.py:536-539`).

Provenance is stamped on every response (`response["compute"]`): cloud success carries `path="cloud_run"`, `job`, `region`, `execution`, `outputs_uri` (`main.py:606-613`); in-process sets `path="in_process"` (`main.py:708`); the `finally` block defaults `path` to `not_run`/`in_process` + `requested` (`main.py:749-757`).

`cloud_available` in `/api/capabilities`

`backend/main.py:347` `capabilities()` returns `deployment.cloud_available = _cloud_available()` (`main.py:379`). `_cloud_available()` (`main.py:489`) is True when **either** `deployment.compute_target == "cloud_run"` (`main.py:499`) **or** (`CLOUD_RUN_JOB` AND

`GOOGLE_CLOUD_PROJECT` env set) AND `storage.output_root` non-empty (`main.py:501-503`). The SPA shows the cloud toggle only when this is True.

How to CONFIGURE a server for cloud serving

Profile (`config.json` `deployment.profile="cloud-serving"` or env `DEPLOYMENT_PROFILE=cloud-serving`). The preset (`backend/config/presets.py:51-56`) sets:
- `deployment.compute_target="cloud_run"`
- `indexing.detector_impl="native"`, `indexing.skip_ai_handoff=False`
- `storage.backend="gcs"`

Applied by the loader via `_deep_merge(defaults, preset, file_data)` (`backend/config/loader.py:167`), with `compute_target` back-filled from the profile if blanked (`loader.py:173-179`). Field defs: `DeploymentConfig.profile`/`compute_target` Literals (`backend/config/models.py:365,368`); `StorageConfig.backend`/`output_root` (`models.py:318-320`).

You must also set `storage.output_root` to a bucket (e.g. `gs://your-bucket`) ? the preset does NOT set it; the dispatch hard-requires it (`main.py:563-566`) and the input must be a cloud URI (`gs://?`), not a local path (`main.py:560-562`).

Env (control plane) consumed by `get_compute_backend` (`backend/compute/__init__.py:63-65`):
- `CLOUD_RUN_JOB` (default `"rally-worker"`)
- `CLOUD_RUN_REGION` (default `"asia-south1"`)
- `GOOGLE_CLOUD_PROJECT` (no default; the GCP project)

Note the README example uses region `asia-south1`/`CLOUD_RUN_JOB=rally-worker`, while `verify_compute_path.py` defaults and `cloudbuild.cr.yaml` use `asia-southeast1` + project `gen-lang-client-0306026826` ? region is owner-environment-specific, not a fixed value.

`indexing.skip_ai_handoff` is forwarded to the cloud job so it yields rallies directly from WASB windows without a Gemini key (`main.py:574-577`); the container is forced INLINE via `deployment.compute_target=local_gpu` + `indexing.detector_impl=native` (`backend/compute/cloud_run.py:41-44`) so it never re-dispatches (guard at `backend/worker/__main__.py:168-170`).

Minimal cloud-serving config:
```
DEPLOYMENT_PROFILE=cloud-serving
CLOUD_RUN_JOB=rally-worker
CLOUD_RUN_REGION=<your-region>
GOOGLE_CLOUD_PROJECT=<your-project>
```

config.json: storage.output_root="gs://<bucket>" (input must be a gs:// URI)

Cloud Run Job deploy recipe (`deploy/cloudrun/`)

`README.md` 4 steps: (1) `gcloud builds submit --tag ?/rally-worker:latest -f deploy/cloudrun/Dockerfile .`; (2) `gcloud run jobs create rally-worker --gpu 1 --gpu-type nvidia-l4 --cpu 4 --memory 16Gi --max-retries 0 --task-timeout 3600`, staging `WASB_WEIGHTS`; (3) export the env above + dispatch via `worker infer` or the API; (4) capture an M7 deploy runlog. The `Dockerfile` (`deploy/cloudrun/Dockerfile`) is `nvidia/cuda:12.4.1-runtime-ubuntu22.04` with a micromamba `wasb` conda env (torch 1.11.0+cu113, ADR-005 BSD channels), the app installed with `.[storage-gcs]`, `ENTRYPOINT ["python3","-m","backend.worker"]`. **WASB-C3 caveat (Dockerfile:14-16):** pretrained weights are NOT baked in (academic-data provenance uncleared); stage at runtime. The untracked `cloudbuild.cr.yaml` is a convenience build config hardcoding the owner's AR path/region.

`scripts/verify_compute_path.py` ? what it asserts

Submits `/api/process` with `--compute local|cloud`, polls `/api/jobs/{id}` to terminal, then asserts `result["compute"]["path"] == --expect` (`in_process|cloud_run`); exits 4 on mismatch (proves no silent fallback). With `--verify-gcp` it shells `gcloud run jobs executions describe <execution>` to confirm the recorded Cloud Run execution exists in GCP (exit 6 if not). Stdlib-only (urllib).

Files
- `C:/Users/avidu/Projects/bhi-computeval/backend/main.py` (`:273`, `:347-381`, `:489-503`, `:506-619`, `:703-708`, `:744-757`)
- `C:/Users/avidu/Projects/bhi-computeval/backend/compute/__init__.py` (`:32-70`)
- `C:/Users/avidu/Projects/bhi-computeval/backend/compute/cloud_run.py`
- `C:/Users/avidu/Projects/bhi-computeval/backend/config/presets.py` (`:51-56`), `backend/config/models.py` (`:311-368`), `backend/config/loader.py` (`:167-187`)
- `C:/Users/avidu/Projects/bhi-computeval/backend/worker/__main__.py` (`:118-170`, `:263-268`, `:496-501`)
- `C:/Users/avidu/Projects/bhi-computeval/deploy/cloudrun/README.md`, `deploy/cloudrun/Dockerfile`
- `C:/Users/avidu/Projects/bhi-computeval/scripts/verify_compute_path.py`
- `C:/Users/avidu/Projects/bhi-computeval/tests/test_api_cloud_dispatch.py` (23 pass)

NOT implemented / honest gaps
- `player_pool` and `max_frames` are **dropped on the cloud path** (not threaded to the worker `infer` CLI) ? `main.py:573` documents this as a follow-up.
- No real Cloud Run execution has been run (M7 owner step pending; `GoogleCloudRunJobClient` is `pragma: no cover`, no deploy runlog exists).
- The whole feature is on an **unmerged branch**; `origin/master` does not yet have the `compute` request field, `cloud_available`, or the verify script.
- WASB weights provenance (WASB-C3) is unresolved ? owner-gated before any public, non-owner cloud serving.

---

spa
I have all the verified facts I need. The async compile poll (#108/#329), compute-picker (#104), wizard (#88), file upload (#66), and locale-to-watermark (#73) are all merged into master and covered by tests. Let me do one final check to confirm the WhatsApp share and download anchor are not gated behind anything, and verify the test suite for upload exists (already saw `useUpload.test.ts`, `upload.test.tsx`). I have enough to report.

Here is the verified, step-by-step user journey.

KhelSutra Studio SPA ? Verified User Journey

All claims below are grounded in `C:/Users/avidu/Projects/khelsutra/web/src` on `master` @ `bc4e7ba` (repo pulled clean, ff-only). The full flow is SHIPPED end-to-end (file upload ? process ? curate ? async build ? preview/download), with one honest caveat about reel preview (below).

Screen layout / orchestrator
`web/src/App.tsx` is the single-page orchestrator. Header (`App.tsx:222-229`) shows the "Khel?Sutra Studio" wordmark + the `LanguageSwitcher`. The body conditionally renders one of two halves based on whether a `videoId` is set (`App.tsx:55-57`, seeded from `?video=` URL param, also settable in-session):

- **No `videoId` ? front-half (ingest):** SetupWizard (first run only) ? IngestPanel.
- **`videoId` set ? back-half (curate):** rallies load, QueryBar/TapFilters, RallyGrid, SelectionFooter, CompileResult.

The two halves are bridged without a reload: `IngestPanel.onLoaded(id)` calls `setVideoId(id)` and rewrites `?video=<id>` via `history.replaceState` (`App.tsx:246-253`).

Step 1 ? First-run setup wizard (once per browser)
`web/src/components/SetupWizard.tsx`. Renders only when `!videoId && !wizardDone` (`App.tsx:242`); a `localStorage` flag `khelsutra.wizard.completed` shows it once (`SetupWizard.tsx:12-20`). It probes `GET /api/capabilities` (`client.ts:184`) and walks 3 honest steps (`SetupWizard.tsx:143-145`):

1. **KeyStep** ? is a Gemini key present? If not, links to `aistudio.google.com/apikey` (GUIDED setup only ? there is NO key-write endpoint; the header comment at `SetupWizard.tsx:1-4` states this explicitly; offline `motion` segmenter works with no key).
2. **ComputeStep** ? shows default segmenter, ffmpeg/ffprobe presence, GPU presence (`SetupWizard.tsx:50-66`).
3. **VideosStep** ? static "ready" copy (`SetupWizard.tsx:68-77`).

Back/Next/Done + Skip buttons (`SetupWizard.tsx:148-176`). If `/api/capabilities` fails, it shows an offline notice + a single "Done" to dismiss (no false capability claims) (`SetupWizard.tsx:95-101, 142, 167-175`).

Step 2 ? Load a video (IngestPanel)

`web/src/components/IngestPanel.tsx`. Two convergent paths:

- **PRIMARY ? in-UI file upload (T4):** a native `<input type="file" accept=".mp4,.mov,...">` styled as a button (`IngestPanel.tsx:140-161`). On pick, `useUpload.start(file)` (`web/src/hooks/useUpload.ts`) chunks the file via `uploadFile()` ? `init` ? sequential `PUT part` ? `complete` (`client.ts:262-291`), honoring the engine's `max_part_mb` (default 95 MB, `client.ts:270`) to stay under ~100 MB edge limits. Shows a **determinate % progress bar** (`IngestPanel.tsx:190-209`), a Cancel button (AbortController-backed), and rejects 0-byte files before hitting the network (`useUpload.ts:38-41`). On completion the returned server **path** is handed to `submit(path, locale, sendCompute)` (`IngestPanel.tsx:52-54`).
- **SECONDARY ? paste a URI:** a text field accepting a `gs://`/`gcs://`/`s3://` bucket URI or a host path (`IngestPanel.tsx:163-185`), submitted to the same back-half.

Both converge on `useIngestJob.submit` (`web/src/hooks/useIngestJob.ts`): `POST /api/process {video_path, locale, compute}` ? `202 {job_id}` (`client.ts:85-99`), then **recursive-setTimeout poll of `GET /api/jobs/{id}`** with 1.5× backoff, 1s?5s, 10-min ceiling (`useIngestJob.ts:15-18, 75-114`). Progress is shown as the engine's **coarse free-text phase words, not a fake %** (`useIngestJob.ts:106`, `IngestPanel.tsx:220-242`). A success-shaped failure (job `done` but `result.status==="failed"` or no `video_id`) is surfaced as `emptyResult`, never silently loaded (`useIngestJob.ts:88-100`). On success `onLoaded(videoId)` fires (`useIngestJob.ts:96-98`).

Compute picker (`IngestPanel.tsx:91-138`): shown ONLY if `capabilities.deployment.cloud_available === true` (`IngestPanel.tsx:37-45`). Two toggle buttons labeled "On my computer" / "In the cloud" (verified i18n: `ingest.compute.local`/`.cloud`). Default = `local` (`IngestPanel.tsx:35`). Picking **cloud** reveals a cost note ("?may cost a little, often just a few cents per match") + a **required acknowledgement checkbox**; until checked, `cloudBlocked` disables both submit surfaces and blocks dispatch ? no paid job runs without explicit consent (`IngestPanel.tsx:50, 57, 61, 67, 117-136`). The choice is sent as `compute` only when the picker is shown, else omitted ? server default (`IngestPanel.tsx:46-47`).

Step 3 ? Rallies load (curate screen appears)

When `videoId` is set, `App.tsx:72-102` calls `listRallies` ? `POST /api/query {video_id}` ? `play_segments` (`client.ts:107-114`). A `ModeBanner` (`App.tsx:289-316`) shows loading / loaded-count / friendly localized error / demo states. On load all rallies are **selected by default** (`App.tsx:85`, `useRallySelection`). Without `?video=`, the app shows 6 hardcoded **DEMO_RALLIES** (`App.tsx:35-42`) and a `mode.demo` banner.

Step 4 ? Find/query rallies

- **QueryBar** (`web/src/components/QueryBar.tsx`): an OFFLINE deterministic parser turns text like "top 15 longest"/"over 10s" into a selection (`QueryBar.tsx:32, 40-43`). A live "parsed-pill" shows how the text was understood; unsupported concepts (shot-type/player/score) render greyed-out as "not available yet" ? never faked (`QueryBar.tsx:66-91`). English example chips ("over 10s", "top 15 longest", ?) show **only for English users** (`QueryBar.tsx:21, 30, 93-107`).
- **TapFilters** (`web/src/components/TapFilters.tsx`): localized tap-only buttons (longest/shortest/over 10s/over 20s/top 5/top 10) for users who can't type English ? each builds the same `ParsedQuery` (`TapFilters.tsx:17-24, 40-57`).
- **Bulk controls** (`App.tsx:263-267`): select all / clear / invert.

Step 5 ? Pick rallies (RallyGrid)

`web/src/components/RallyGrid.tsx`: one selectable card per rally showing **honest fields only** ? stable chronological ordinal, mm:ss start, rally **length** (the trusted axis). A shot-count chip appears only when present and is flagged experimental; `server`/`receiver`/`events` (motion fabrications) are never rendered (`RallyGrid.tsx:1-5, 57-72`). Tap toggles selection (`App.tsx:208-211`).

Step 6 ? Build the reel (async, #108/#329)

SelectionFooter (`web/src/components/SelectionFooter.tsx`, sticky) shows selection count/total, a reel-name input, and the Build button. Build is disabled when count=0, building, or all selected rallies are below the engine's `min_shot_count` floor (`SelectionFooter.tsx:23`). An **honest floor warning** surfaces rallies the engine would silently drop, before build (`SelectionFooter.tsx:27-51`, fed by `floorRisk` from `GET /api/config` `min_shot_count`, `App.tsx:106-116`).

`App.onBuild` (`App.tsx:126-162`) ? `compileReel(videoId, ids, filename, cid, locale)`. This is now **ASYNC** (`client.ts:127-171`): `POST /api/compile` returns `202 + job_id`, then `pollCompileJob` polls `GET /api/jobs/{id}` (1s?5s backoff, 10-min ceiling) until the ffmpeg stitch finishes ? deliberately to avoid the ~100s edge 524 a long stitch would otherwise hit (`client.ts:116-125, 146-171`). The **active UI locale is sent** so the engine localizes the reel's watermark (`App.tsx:150`, LW1). A failed/timed-out/no-URL compile throws and is mapped to friendly localized text (`client.ts:153-167`, `App.tsx:154-161`).

Step 7 ? Preview + download

CompileResult (`web/src/components/CompileResult.tsx`): a "building?" card with spinner+reassurance during the stitch (`CompileResult.tsx:26-33`). On success:
- An inline **`<video controls playsInline>`** previews the reel via `highlightUrl(videoId, filename)` ? `GET /api/highlights/{video_id}/{filename}` (`CompileResult.tsx:61-67`, `client.ts:174-176`).
- A **Download** anchor (`<a download>`) (`CompileResult.tsx:69-75`).
- A **WhatsApp share** button (`wa.me` with localized pre-filled text; it can't attach the local file, so the user attaches the just-downloaded reel ? comment at `CompileResult.tsx:76-86`).

On failure it shows the engine's reason verbatim, never swallowed (`CompileResult.tsx:89-93`).

Localization ? 6 locales VERIFIED

`web/src/i18n/index.ts`: `SUPPORTED_LOCALES = ["en","kn","hi","es","da","id"]` (`index.ts:25`). All 6 catalogs exist at repo-root `locales/{en,kn,hi,es,da,id}/common.json` (verified on disk) and are imported (`index.ts:16-43`). Negotiation order: persisted pref ? `?lang=` ? browser ? `en` fallback (`index.ts:58-64`); `<html lang>` kept in sync (`index.ts:70-76`). **LanguageSwitcher** (`web/src/components/LanguageSwitcher.tsx`) renders all 6 as autonym buttons in the header. Kannada/Hindi ship self-hosted Noto fonts (`web/src/main.tsx:6-8`).

Caveats / what is NOT as one might assume (verified)

- **Reel preview depends on browser codec support.** The `<video>` src is the raw `/api/highlights/...` stream; nothing transcodes for browser playback. Download always works regardless.
- **Gemini key entry is guided, not in-app.** The wizard links out to get a key; there is no key-write endpoint ? the engine reads it from its own env (stated at `SetupWizard.tsx:1-4`).
- **The query parser is English-only.** Non-English users get TapFilters instead of the QueryBar's example chips (`QueryBar.tsx:30, 93`); this is by design, not a gap.
- **Demo mode is data-only.** Without `?video=`, six hardcoded demo rallies render and Build will fail with a localized "no video" message (`App.tsx:131-137`) ? demo rallies have no real `video_id` to compile.
- **`server`/`receiver`/`events` are never surfaced** anywhere in the UI (motion-segmenter fabrications), per the type annotations (`api/types.ts:20-26`) and RallyGrid honesty rule.

Relevant files (all absolute):
- `C:/Users/avidu/Projects/khelsutra/web/src/App.tsx`
- `C:/Users/avidu/Projects/khelsutra/web/src/components/IngestPanel.tsx`
- `C:/Users/avidu/Projects/khelsutra/web/src/hooks/useUpload.ts`
- `C:/Users/avidu/Projects/khelsutra/web/src/hooks/useIngestJob.ts`

- `C:/Users/avidu/Projects/khelsutra/web/src/api/client.ts`
- `C:/Users/avidu/Projects/khelsutra/web/src/api/types.ts`
- `C:/Users/avidu/Projects/khelsutra/web/src/components/CompileResult.tsx`
- `C:/Users/avidu/Projects/khelsutra/web/src/components/SelectionFooter.tsx`
- `C:/Users/avidu/Projects/khelsutra/web/src/components/SetupWizard.tsx`
- `C:/Users/avidu/Projects/khelsutra/web/src/components/QueryBar.tsx`
- `C:/Users/avidu/Projects/khelsutra/web/src/components/TapFilters.tsx`
- `C:/Users/avidu/Projects/khelsutra/web/src/components/RallyGrid.tsx`
- `C:/Users/avidu/Projects/khelsutra/web/src/components/LanguageSwitcher.tsx`
- `C:/Users/avidu/Projects/khelsutra/web/src/i18n/index.ts`
- `C:/Users/avidu/Projects/khelsutra/locales/{en,kn,hi,es,da,id}/common.json`

---

docs

I have all verified facts. Note `docs/README.md` is a flat doc index (no top-level title/intro heading ? it begins mid-list at line 1 with `RALLY_QUALITY_RESEARCH.md`), and the root README is governance-listed as a LIVING doc in `DOC_STATUS.md:73`. Here is my recommendation.

Recommendation

Is there an existing end-user install+usage guide?

Partially ? in the root `README.md`, not in `docs/`. There is **no** `QUICKSTART` / `USER_GUIDE` / `getting-started` doc anywhere under `docs/` (verified: filename grep for those patterns returns nothing; the only content hits for "quickstart/user guide" are in strategy docs `PLATFORM_ARCHITECTURE.md`, `COMMERCIALIZATION.md`, `I18N_PLAN.md`, and archived vendoring docs ? none are end-user reel guides).

What *does* cover install+usage for end users today is the repo-root **`README.md`** (449 lines, `README.md:1-449`). It already has:
- An explicit **"Batteries-included install (LIGHT tier ? recommended for end users)"** section (`README.md:35-65`) covering `uv tool install` / `scripts/install.py` ? `indexer --app` (CLI flag verified to exist at `backend/main.py:1139`), app-data state dirs, ffmpeg caveat, and clean uninstall.
- **"Running the Application"** (`README.md:120-181`) ? launching the web dashboard (`indexer --server`, flag verified `backend/main.py:1138`), CLI processing mode, and reel/trim/watermark usage.
- A 14-question **FAQ** (`README.md:254-359`) and config-tuning reference (`README.md:182-253`).

But it is a mixed-audience doc ? heavily weighted toward developers/power-users (editable `pip install -e .`, PyTorch wheel indices, `--debug-clip`, `config.json` threshold tuning, the test suite, project structure). It is **not** a clean, task-first "non-technical user creates a highlight reel" walkthrough, and the actual friendly reel-creation flow is the **SPA**, which lives in the *sibling* `khelsutra` repo, not this engine repo (`UI_PROPOSAL.md:6-10` confirms the alpha SPA moved to `khelsutra/web/` and the non-tech UX is tracked in `khelsutra/docs/FRIENDLY_AND_MULTILINGUAL.md`). So this engine repo has no dedicated end-user guide for the reel-creation user journey.

Recommended placement for the new guide

Create a new doc: `C:/Users/avidu/Projects/bhi-computeval/docs/CREATING_REELS.md` (a task-oriented "install ? process a video ? pick rallies ? get a watermarked reel" guide for end users via `indexer --app`).

- Prefer `CREATING_REELS.md` over `USER_GUIDE.md`: it is action/outcome-named (matches this repo's doc-naming style ? `PACKAGING_PLAN`, `SETUP_NEW_MACHINE`, `CLOUD_GPU_DRYRUN_GUIDE`) and avoids colliding conceptually with the root README, which is already the catch-all "user guide."
- Do **not** name it `QUICKSTART.md` if it will be more than a few steps; reserve that only if you want a deliberately short page.

Should an existing doc be updated/linked instead?

- **Do not duplicate** the root `README.md` install section ? **link to it.** The new guide

should reference the root README's "Batteries-included install" (`README.md:35-65`) for install mechanics and focus on the *usage journey* (open the app, point at a video, review/pick rallies, download the reel), not re-derive install.
- **Scope check (verified, not aspirational):** `indexer --app` ships and boots the bundled SPA single-origin (`backend/main.py:1139`; `PACKAGING_PLAN.md:12` "v1 LIGHT is FEATURE-COMPLETE"). The **native-GPU (WASB) and cloud serving tiers are owner-only / not in the default download** (`README.md:65`, `PACKAGING_PLAN.md:18,23`) ? the guide must say the default LIGHT tier is Gemini/`motion`/cpu-mock only and NOT promise on-device GPU quality or one-click cloud. **ffmpeg is a manual prerequisite** (`README.md:61,67-74`) ? not auto-installed; call this out as a required step.
- Cross-repo note: the friendly multilingual reel UX itself is implemented in the sibling `khelsutra` repo (`UI_PROPOSAL.md:6-10`). If the guide documents on-screen SPA steps, flag that the UI is bundled from there so screenshots/wording stay in sync.

How `docs/README.md` indexes docs (so the new one is added consistently)
`docs/README.md` is a **flat, single-level bulleted index grouped under `##` section headings** (verified `README.md:1-44`). Each entry is one bullet of the exact form:

```
- [FILENAME.md](FILENAME.md) ? **<bold status/tag>** one-paragraph description (what it is, status, source-of-truth pointer).
```

Sections present: *(top, untitled list)* ? `## ? Documentation status & governance` ? `## Product / platform plans` ? `## Practical / ongoing` ? `## Data layer ?` ? `## Conventions & templates` ? `## Historical documents`. There is **no H1/intro** ? the file opens directly at line 1 with the `RALLY_QUALITY_RESEARCH.md` bullet.

Add the new guide under `## Practical / ongoing` (alongside `SETUP_NEW_MACHINE.md` at `README.md:23`, which is the dev/ops runbook ? the new doc is its end-user counterpart), as e.g.:
`- [CREATING_REELS.md](CREATING_REELS.md) ? **? End-user guide:** install the LIGHT-tier app (? root README §0) ? indexer --app ? process a video ? pick rallies ? download a watermarked reel. LIGHT tier only (Gemini/motion/cpu-mock; no on-device GPU/cloud). Pairs with PACKAGING_PLAN.md.`

Also: governance convention per `CODE_MAP.md:33` ("A doc ? Index it in `docs/README.md`") and `DOC_STATUS.md:21` ? when you add it, list it in `docs/README.md` and add a row/mention to `docs/DOC_STATUS.md` (the README itself is already tracked there as LIVING at `DOC_STATUS.md:73`).

Verification caveat
`git pull --ff-only` **failed** ? the local checkout is on branch `feat/compute-provenance` whose remote ref wasn't fetched (HEAD `ee6fba8`), so these facts reflect the **local checkout, which may lag/lead `origin/master`**. All `file:line` citations above were read from the working tree; re-verify the README line ranges against `origin/master` before writing, since the root README is a LIVING doc that moves between sessions.

B) DRAFT:
I've written the user guide. Here is the markdown.

```markdown
# Creating Highlight Reels with KhelSutra

*A friendly guide for coaches and players. Install the app, point it at a match video, pick the best rallies, and download a watermarked highlight reel.*

> This guide covers the **LIGHT tier** ? the batteries-included downloadable app
> you run on your own computer. It uses the offline `motion` detector by default
> (no GPU needed). On-device GPU (native WASB) and one-click cloud serving are
> **advanced / owner-provisioned tiers**, not part of the default download ? see
> [Run it in the cloud](#6-run-it-in-the-cloud-advanced--optional).
```

---

1. What it does

KhelSutra watches a badminton match video, automatically finds the **rallies** (the moments when play is actually happening), and lets you stitch the ones you pick into a single **highlight reel** with a watermark in your language. You run it locally on your own machine through a small web app in your browser ? no SSH, no command-line gymnastics beyond one command to start it.

---

2. Prerequisites

You need two things installed first.

Python 3.10 or newer

Get it from [python.org](https://www.python.org/downloads/) if you don't have it.

FFmpeg (and FFprobe)

FFmpeg is required to read and cut video, and it is **not bundled** with the app (licensing reasons) ? you install it once yourself:

Your OS	Command
Windows	`winget install Gyan.FFmpeg`
macOS	`brew install ffmpeg`
Linux	`sudo apt-get install -y ffmpeg`

The app finds `ffmpeg`/`ffprobe` on your `PATH` automatically. (Advanced: you can override the locations with the `RALLY_FFMPEG` and `RALLY_FFPROBE` environment variables.)

> **Tip:** after installing, run `indexer --setup` (see below) to confirm FFmpeg
> is detected ? it prints the same install hints if anything is missing.

---

3. Install & run

Install

The easiest path uses [`uv`](https://docs.astral.sh/uv/):

```bash
uv tool install .
uv tool update-shell   # adds the 'indexer' command to your PATH (run once)
```

Then open a **new terminal** so the `indexer` command is found.

Alternatively, use the guided installer, which also creates a desktop launcher shortcut and an uninstaller for you:

```bash
python scripts/install.py
```

> **Note:** the package is **not yet published to PyPI**, so install from a local
> checkout (`uv tool install .`) or the installer script ? a bare
> `uv tool install badminton-highlight-indexer` won't resolve yet.
>
> *Developer path:* `pip install -e .` (or `pip install -e .[dev]`) from a source

> checkout also registers the `indexer` command.

Run

```bash
indexer --app
```

This starts the app **on your own computer only** (localhost), picks a free port automatically, and opens the web app in your default browser once it's ready.

(To start it without auto-opening a browser, set `RALLY_NO_BROWSER=1`.)

First-run setup

The first time the web app opens, a short **3-step setup wizard** appears (it shows once per browser):

1. **AI key (optional)** ? KhelSutra can use a Google **Gemini** key for higher quality. If you don't have one, the wizard links you to aistudio.google.com/apikey. **You don't need a key** ? the offline `motion` detector works with no key at all. *(There is no in-app box to paste the key; if you set one up, the app reads it from its own environment.)*
2. **Compute** ? shows your default detector, whether FFmpeg is detected, and whether a GPU is present.
3. **Videos** ? confirms you're ready to go.

You can **Skip** the wizard at any time.

---

4. Create a reel ? the easy way (web app)

This is the recommended flow for most people.

1. **Open the app** (`indexer --app`).
2. **Add your video.** On the start screen you can either:
   - **Upload a file** ? click the upload button and pick an `.mp4` or `.mov` match video. A progress bar shows the upload (large files upload in chunks; you can Cancel mid-upload).
   - **Paste a URI** ? point at a cloud/bucket path (e.g. `gs://...`) or a file path instead.
3. **Wait for rally detection.** Processing runs in the background and **can take a few minutes**. The app polls automatically and shows the current phase ? no need to refresh. When it finishes, your rallies appear.
4. **Find and pick rallies.**
   - All rallies start **selected** by default.
   - Use the **search box** to type things like `top 15 longest` or `over 10s` (English), or use the **tap filters** (longest / shortest / over 10s / over 20s / top 5 / top 10) ? handy in any language.
   - Each rally card shows its start time and length. Tap a card to toggle it.
   - Use **select all / clear / invert** for bulk changes.
5. **Build the reel.** Give it a name and click **Build**. This also runs in the background and **can take a few minutes** while FFmpeg stitches your clips; the app polls until it's done. The reel's watermark is written in your current app language.
6. **Preview & download.** When it's ready you get an inline video preview, a **Download** button, and a **WhatsApp share** button (WhatsApp can't attach the file directly, so download it first, then attach the downloaded reel).

> **Choosing a compute location:** if your server has cloud serving available, a
> small **"On my computer" / "In the cloud"** toggle appears before you process.
> Cloud may cost a little (often just a few cents per match) and requires ticking

> a consent checkbox first. If you don't see the toggle, everything simply runs on
> your computer.

---

5. Create a reel ? the API way (advanced / scripting)

For automation, the same flow is a chain of HTTP calls. Endpoints below are
served by `indexer --app` (or `indexer --server`). Examples use `curl` against
`http://127.0.0.1:8000`.

Step 1 ? Check capabilities

```bash
curl http://127.0.0.1:8000/api/capabilities
```

Returns available `segmenters`, the `default_segmenter`, FFmpeg/GPU presence,
whether a Gemini key is present, and `deployment.cloud_available`.

Step 2 ? Get your video onto the server

Option A ? upload a local file (chunked, parts must be sent in order):

```bash
# 1. init
curl -X POST http://127.0.0.1:8000/api/upload/init \
  -H 'Content-Type: application/json' \
  -d '{"filename":"match.mp4"}'
# -> {"upload_id": "...", "max_part_mb": 95, "next_part": "/api/upload/<id>/part/0"}

# 2. upload each part (raw bytes), strictly sequential: part/0, part/1, ...
curl -X PUT --data-binary @part0.bin \
  http://127.0.0.1:8000/api/upload/<upload_id>/part/0

# 3. complete -> returns the server-side 'path' to use next
curl -X POST http://127.0.0.1:8000/api/upload/<upload_id>/complete \
  -H 'Content-Type: application/json' \
  -d '{"total_parts": 1}'
# -> {"path": "<server path>", "video_id": "...", ...}
```

Option B ? skip upload and pass a `gs://` / Drive URI directly to Step 3 as
`video_path`.

Step 3 ? Process the video (async: returns 202, then poll)

```bash
curl -X POST http://127.0.0.1:8000/api/process \
  -H 'Content-Type: application/json' \
  -d '{"video_path": "<server path or gs:// URI>"}'
# -> 202 {"job_id": "...", "status": "queued", "poll": "/api/jobs/<job_id>"}
```

Optional body fields: `segmenter`, `locale`, `compute` (`"local"`/`"cloud"`),
`player_pool`, `max_frames`.

Poll until done:

```bash
```

```
curl http://127.0.0.1:8000/api/jobs/<job_id>
# status: queued | running | done | failed
# on success, result.status == "success" and result.play_segments are available
```

Step 4 ? Query the detected rallies

```bash
curl -X POST http://127.0.0.1:8000/api/query \
  -H 'Content-Type: application/json' \
  -d '{"video_id": "<video_id>"}'
# -> {"play_segments": [{"id": 0, "start_time": ..., "end_time": ..., "metadata": {...}}, ...]}
```

This call is synchronous. Optional filters: `server`, `receiver`, `duration_min`, `event_type`. Collect the `id`s of the segments you want.

Step 5 ? Compile the reel (async: returns 202, then poll)

```bash
curl -X POST http://127.0.0.1:8000/api/compile \
  -H 'Content-Type: application/json' \
  -d '{"video_id": "<video_id>", "segment_ids": [0,2,5], "output_filename": "highlights.mp4"}'
# -> 202 {"job_id": "...", "status": "queued", "poll": "/api/jobs/<job_id>"}
```

Optional: `locale` (localizes the watermark text + font). Poll the job until `done`; the job `result` then contains a `download_url`.

Step 6 ? Download

```bash
curl -O http://127.0.0.1:8000/api/highlights/<video_id>/highlights.mp4
```

(`download_url` from the compile job points here.)

---

6. Run it in the cloud (advanced / optional)

By default everything runs **on your computer**. A cloud option exists for offloading the heavy detection work to a GPU.

- **The picker:** when cloud serving is configured, the web app shows an **"On my computer" / "In the cloud"** toggle (and the API accepts `"compute": "cloud"` on `/api/process`). It only appears when `cloud_available` is `true` in `/api/capabilities`.
- **What `cloud_available` means:** the server is configured for cloud serving ? either its deployment profile is `cloud-serving`, or the required Cloud Run environment variables and an output bucket are set.
- **Configuring it (developer/owner step):** set the `cloud-serving` deployment profile, a `storage.output_root` bucket (e.g. `gs://your-bucket`), and the `CLOUD_RUN_JOB`, `CLOUD_RUN_REGION`, and `GOOGLE_CLOUD_PROJECT` environment variables. Inputs must be cloud URIs (`gs://...`), not local paths.
- **Deploy recipe:** see `deploy/cloudrun/README.md` for the full Cloud Run Job build/create steps (the heavy infra lives there).
- **Prove the path works:** `scripts/verify_compute_path.py` submits a job with `--compute local|cloud`, polls to completion, and asserts which backend actually ran ? so you can confirm a cloud request really ran in the cloud (no silent fallback to local).

> **Honest scope note:** the cloud path is built and tested with mocks, but a
> real production Cloud Run GPU run is an owner-provisioned step that may not be
> set up on your install. If you don't see the toggle, cloud isn't available ?
> just run on your computer.

---

7. Troubleshooting

"FFmpeg not found" / processing fails immediately.
FFmpeg is a manual prerequisite (it is not auto-installed). Install it per
[Prerequisites](#2-prerequisites), then run `indexer --setup` to confirm it's
detected.

I don't have a Gemini AI key ? does anything still work?
Yes. The offline **`motion`** detector is the default and needs no key, no GPU,
and no internet. A Gemini key (the `gemini` segmenter) is optional and only used
if you set one up. `/api/capabilities` lists exactly which segmenters your
install has available.

A job shows `failed`.
Check the job's `error` and `result.status` via `GET /api/jobs/<job_id>`. The app
never silently loads a failed result ? it surfaces the engine's reason. Common
causes: a missing/unreadable video file (you'll get a clear 404/400 at submit
time), an unknown `segmenter` name, or no rallies meeting the minimum length
floor when compiling.

The reel won't preview in the browser (but downloads fine).
The preview plays the raw reel stream and relies on your browser's video codec
support ? nothing transcodes it for playback. If preview is blank, just use the
Download button; the file is fine.

Building used to time out around ~100 seconds (Cloudflare 524).
That's fixed. Both processing (`/api/process`) and reel building (`/api/compile`)
are now **asynchronous** ? they return a job ID immediately and the app polls in
the background, so long stitches no longer hit the edge timeout. Just wait; the
UI keeps polling.

"Another instance is already running."
The app holds a single-instance lock per data directory ? close the other
running copy, or start the new one against a different output directory.

---

*Install mechanics live in the root [`README.md`](../README.md) (§ Batteries-included
install). The default LIGHT tier is offline `motion` / optional Gemini only ? it
does not promise on-device GPU quality or one-click cloud. See
[`PACKAGING_PLAN.md`](PACKAGING_PLAN.md) for the packaging roadmap.*
```

Saved to `C:\Users\avidu\Projects\bhi-computeval\docs\CREATING_REELS.md`.

#### 2. assistant (2026-06-22T18:54:27.607Z)

I'll verify the draft against the facts, checking commands, endpoints, flags, and the load-
bearing async behavior. Let me cross-check a few specifics in the facts first.

The facts are self-contained, so I'll work directly from them. Here is my prioritized fix list.

---

# Fix list ? `CREATING_REELS.md` draft

The draft is strong and largely accurate. The async behavior (process + compile both 202+poll)
```

is correctly represented throughout. Found one real fabrication, two missing load-bearing steps, and several precision/clarity fixes.

P0 ? Fabrication (command not supported by the facts)

1. `curl -O .../api/highlights/<video_id>/highlights.mp4` uses a flag the facts never establish, and the bigger issue: the path filename must equal `output_filename`.
Not a fabricated *endpoint* (the endpoint is real), but the draft's Step 6 is subtly wrong in a way that will break for users. The facts state `filename` in the download path "= the bare name passed to `/api/compile`" (the `output_filename`). The draft happens to reuse `highlights.mp4` consistently, so this is OK ? but add an explicit note so a user who changes the name knows the download path must match. **Correction:** add "the final path segment must be exactly the `output_filename` you passed to `/api/compile`." Keep `curl -O` (valid). No actual fabricated flag ? downgrade severity, but the matching constraint is load-bearing.

P1 ? Missing load-bearing steps

2. Step 5 (API "Build") omits the `min_shot_count` floor fast-fail ? users will get a confusing 400.
The facts: `/api/compile` fast-fails at submit with 400 if "all segments below `stitching.min_shot_count` floor" or "no matching segments." The draft's troubleshooting mentions the floor only for compiling in passing, but the API Step 5 gives no warning. **Correction:** add a note under API Step 5: "If every chosen segment is below the engine's minimum shot-count floor (or none of the `segment_ids` match), `/api/compile` returns 400 at submit ? pick more/longer rallies."

3. Step 2 Option A (chunked upload) is missing the per-part size constraint that makes "parts in order" actually work.
The facts: parts are capped at `max_part_mb` (default 95 MB) and the *whole* upload at `max_upload_mb` (default 4096). The draft shows `max_part_mb: 95` in the init response comment but never tells the user they must split their file into ?95 MB chunks themselves. A user sending one 500 MB `part0.bin` gets a 413. **Correction:** add "Split your file into parts no larger than `max_part_mb` (default 95 MB) before uploading; send them strictly in order (`part/0`, `part/1`, ?). Total upload limit defaults to 4096 MB."

4. The `/api/compile` request in API Step 5 is missing nothing required ? but the draft never says `output_filename` is validated for safety (400 on unsafe name).
Minor, but a user passing a path-like name will get a 400. **Correction (optional):** note "use a plain filename like `highlights.mp4` ? paths/unsafe names are rejected with 400."

P2 ? Accuracy / precision

5. Section 3 Install: `uv tool install .` correctly requires being *in a checkout*, but the draft never says so.
The facts are explicit that today's install is "from a local checkout (`.`)". The draft says "install from a local checkout" in the PyPI note, but the `uv tool install .` block at the top gives no hint the user must first obtain and `cd` into the source. **Correction:** preface the install block with "From inside a clone of the repository:" so the bare `.` resolves.

6. Wizard Step 2 label. Draft calls it "**Compute**"; the facts/SPA call the step **ComputeStep** showing "default segmenter, ffmpeg/ffprobe presence, GPU presence." Fine as-is ? but "default detector" vs the facts' "default segmenter" is a harmless rewording. No change needed; flagging only that "GPU is present" is a heuristic (`nvidia-smi` on PATH), not a real CUDA check. **Optional correction:** "whether a GPU appears present" to avoid overpromising.

7. `/api/query` optional filters list is accurate (`server`, `receiver`, `duration_min`, `event_type`) ? matches facts. No change. But note the facts say `server`/`receiver`/`events` are motion fabrications never surfaced in the SPA; the API guide exposing them as filters is acceptable for the "advanced" audience. No change required.

P3 ? Clarity for non-technical users

**8. Section 5 intro says endpoints are "served by `indexer --app` (or `indexer --server`)" ?

true, but `--server` was never introduced to a non-technical reader.** Minor. Consider dropping the `--server` aside from the user-facing section, or one line: "(`--server` is the headless variant for advanced/network setups)."

9. The "WhatsApp can't attach the file directly" explanation is good and matches the facts. No change.

10. Cloud section correctly labels everything advanced/owner and matches the "mock-tested, real Cloud Run not yet run" caveat. Accurate. No change.

Things verified CORRECT (no fabrications found)

- Both `/api/process` and `/api/compile` are async 202+poll ? correctly stated everywhere, including troubleshooting. ?
- `/api/query` is synchronous ? correctly stated. ?
- `indexer --app` is localhost-only, auto-free-port, auto-browser, `RALLY_NO_BROWSER=1` to suppress. ?
- `uv tool update-shell`, not-on-PyPI caveat, `python scripts/install.py`, `pip install -e .[dev]` dev path. ?
- FFmpeg manual prereq + exact per-OS install commands + `RALLY_FFMPEG`/`RALLY_FFPROBE` + `indexer --setup`. ?
- Python ? 3.10. ?
- Default `motion` detector, no key/GPU/cloud needed; Gemini optional, no in-app key box (guided link only). ?
- Compute picker gated on `cloud_available`; cloud consent checkbox; "On my computer"/"In the cloud" labels. ?
- Single-instance lock per data dir. ?
- `cloud_available` definition, `cloud-serving` profile, `storage.output_root`, `CLOUD_RUN_JOB`/`CLOUD_RUN_REGION`/`GOOGLE_CLOUD_PROJECT`, `verify_compute_path.py`, `deploy/cloudrun/README.md`. ?
- Cloudflare 524 / ~100s rationale for async. ?

Net: no invented endpoints or flags. The most important real gaps are the **chunked-upload per-part split (#3)** and the **`min_shot_count`/no-matching-segments 400 at compile (#2)** ? both are load-bearing steps an API user will hit. Fix those two first; the rest are precision polish.